

Sprint 10 Full Structured Notes: Prop Drilling, State Management, Context API, Redux, Zustand, and React DevTools

0. Where You Are Before Sprint 10

Before Sprint 10, you should already understand these React ideas:

- React is a JavaScript library for building user interfaces.
- Vite helps you create and run React projects.
- JSX lets you write UI-like syntax inside JavaScript.
- Components are reusable pieces of UI.
- Props pass data from a parent component to a child component.
- `children` lets one component wrap other JSX.
- Conditional rendering lets React show different UI based on data.
- `map()` renders lists.
- Stable keys help React track list items.
- Events let React respond to clicks, typing, and form submissions.
- Controlled inputs store form values in React state.
- `useState` lets React remember changing data.
- State updates should be immutable.
- Object state should be copied before changing one field.
- Array state should be updated with safe methods like spread, `filter()`, and `map()`.
- `useReducer` can organize related state logic.
- `useRef` can access DOM elements or store values that should not cause re-renders.
- `useEffect` runs side effects after React renders.
- Cleanup functions stop old timers, listeners, and subscriptions.
- Custom hooks let you reuse hook logic.
- Context API lets you share app-level values like user, theme, or language.
- Data fetching lets React communicate with APIs or backend services.
- Loading, data, and error states make async UI clear.
- `useMemo`, `useCallback`, and `React.memo` help avoid unnecessary work in some cases.
- React Router connects URLs to React components.
- Dynamic routes, nested routes, 404 routes, protected routes, and lazy-loaded routes help make a React app feel like a multi-page application.

Sprint 10 starts after you can build a working React app with multiple screens.

Now the main problem changes.

Earlier sprints focused on this question:

How do I build components and make them work?

Sprint 10 focuses on this question:

How do I organize shared state and debug a larger React app?

1. Sprint 10 Goal

The goal of Sprint 10 is to understand how state should be shared and debugged in larger React apps.

By the end of Sprint 10, you should be able to:

- explain what prop drilling is,
- explain why prop drilling becomes hard in larger apps,
- explain the idea of single source of truth,
- know when local state is enough,
- know when to lift state up,
- know when Context API is useful,
- know when Context API is not enough,
- understand what Redux is for,
- understand what Redux Toolkit is for,
- understand what RTK Query is for,
- understand what Zustand is for,
- use React DevTools to inspect components,
- use the Components tab to inspect props, state, and context,
- use the Profiler tab to investigate re-renders,
- debug prop mismatch problems.

Simple version:

Sprint 10 teaches you how to share state cleanly and debug React apps when the component tree becomes larger.

2. Big Mental Model

In small React apps, this is usually enough:

```
useState  
props  
local component state
```

Example:

```
function Counter() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <button type="button" onClick={() => setCount(count + 1)}>  
      Count: {count}  
    </button>  
  );  
}
```

This is fine because only one component needs the `count` value.

But larger apps have shared data.

Examples of shared data:

- logged-in user,
- theme mode,
- language,
- cart count,
- current organization,
- global notification state,
- API/server data,
- dashboard filters,
- selected project,
- permissions.

When many components need the same data, you need a clear plan.

Big Sprint 10 idea:

Small app:
Local state and props are usually enough.

Larger app:
Shared state needs a clear owner.

Complex app:

You may need Context, Redux Toolkit, RTK Query, Zustand, or another state tool.

3. The Problem Sprint 10 Solves

Imagine this component tree:

```
App
├── Layout
│   ├── Header
│   │   └── Navbar
│   │       └── UserMenu
```

Suppose `UserMenu` needs the logged-in user.

The `user` state lives in `App`:

```
function App() {
  const user = { name: "Sangeeth" };

  return <Layout user={user} />;
}
```

But `UserMenu` is far below `App`.

Without Context, you may need to pass `user` like this:

```
App -> Layout -> Header -> Navbar -> UserMenu
```

Problem:

```
Layout does not use user.
Header does not use user.
Navbar may not use user.
They only pass user downward.
```

This is called prop drilling.

4. Prop Drilling

4.1 What Is Prop Drilling?

Simple definition:

Prop drilling means passing props through components that do not use them, only so a deeper component can receive them.

Example:

```
function App() {  
  const user = { name: "Sangeeth" };  
  
  return <Layout user={user} />;  
}  
  
function Layout({ user }) {  
  return <Header user={user} />;  
}  
  
function Header({ user }) {  
  return <Navbar user={user} />;  
}  
  
function Navbar({ user }) {  
  return <UserMenu user={user} />;  
}  
  
function UserMenu({ user }) {  
  return <p>Logged in as {user.name}</p>;  
}
```

This code works.

But it is not clean when the app grows.

`Layout`, `Header`, and `Navbar` are carrying data they do not need.

4.2 Prop Drilling Mental Model

```
App owns data
↓
Layout receives data but does not use it
↓
Header receives data but does not use it
↓
Navbar receives data but does not use it
↓
UserMenu finally uses data
```

The deeper the component tree becomes, the more painful this becomes.

4.3 Why Prop Drilling Becomes Hard

Prop drilling becomes hard because:

```
More levels = more repeated prop passing
More props = more messy component function parameters
Renaming one prop = many files may need changes
Moving one component = prop path may break
Debugging = harder to trace where data came from
```

Example of messy prop drilling:

```
function Layout({ user, theme, cartCount, logout, toggleTheme }) {
  return (
    <Header
      user={user}
      theme={theme}
      cartCount={cartCount}
      logout={logout}
      toggleTheme={toggleTheme}
    />
  );
}
```

This component may not use any of these values itself.

It is only acting like a pipe.

That is the warning sign.

4.4 Is Prop Drilling Always Bad?

No.

Small prop passing is normal React.

This is fine:

```
function App() {  
  return <UserCard name="Sangeeth" />;  
}  
  
function UserCard({ name }) {  
  return <h2>{name}</h2>;  
}
```

This is also fine:

```
Parent -> Child
```

Prop drilling becomes a problem when data passes through many components that do not use it.

Beginner rule:

```
Passing props 1 or 2 levels is normal.  
Passing props through many uninterested components is prop drilling.
```

5. Single Source of Truth

5.1 What Is Single Source of Truth?

Simple definition:

A single source of truth means shared data should have one clear owner.

Bad mental model:

```
Navbar has its own user
Dashboard has its own user
Profile has its own user
Settings has its own user
```

Problem:

```
Navbar may think the user is logged in.
Dashboard may think the user is logged out.
Profile may show old user data.
Settings may show different user data.
```

This creates inconsistent UI.

Better mental model:

```
One shared user state
↓
Navbar reads it
Dashboard reads it
Profile reads it
Settings reads it
```

Now the app has one clear owner for the user data.

5.2 Why Single Source of Truth Matters

Single source of truth helps because:

- the app is easier to reason about,
- data is less likely to become inconsistent,
- updates happen in one place,
- debugging is easier,
- many components can read the same current value.

Example:

```
Logged-in user changes
↓
One user state updates
```


↓

Navbar, Dashboard, Profile, and Settings all update from the same data

5.3 Where Can the Single Source of Truth Live?

In React, the single source of truth can live in different places.

Examples:

- Local parent state
- Context Provider
- Redux store
- Zustand store
- Server cache / data-fetching store

The correct choice depends on:

- How many components need the data?
- How far apart are those components?
- How often does the data change?
- How complex are the update rules?
- Is the data client state or server state?

6. Local State, Lifted State, and Shared State

Before jumping into Context or Redux, you should choose the simplest correct tool.

6.1 Local State

Use local state when only one component needs the data.

Example:

```
import { useState } from "react";

function SearchInput() {
  const [query, setQuery] = useState("");
```

```

    return (
      <input
        value={query}
        onChange={(event) => setQuery(event.target.value)}
        placeholder="Search..."
      />
    );
  }
}

```

This is local state.

Only `SearchInput` needs `query`.

Beginner rule:

If only one component needs the data, keep the state local.

6.2 Lifting State Up

Sometimes two sibling components need the same state.

Example tree:

```

App
├── SearchInput
└── SearchPreview

```

If both need `query`, put `query` in their nearest shared parent.

```

import { useState } from "react";

function SearchInput({ query, onQueryChange }) {
  return (
    <input
      value={query}
      onChange={(event) => onQueryChange(event.target.value)}
      placeholder="Search..."
    />
  );
}

```

```
function SearchPreview({ query }) {
  return <p>You searched for: {query}</p>;
}

export default function App() {
  const [query, setQuery] = useState("");

  return (
    <main>
      <h1>Search</h1>
      <SearchInput query={query} onQueryChange={setQuery} />
      <SearchPreview query={query} />
    </main>
  );
}
```

Simple definition:

Lifting state up means moving state to the nearest parent that needs to share it.

Beginner rule:

If sibling components need the same data, lift state to their nearest common parent.

6.3 Shared App State

Sometimes data is needed by many distant components.

Example:

```
App
├─ Navbar needs user
├─ Dashboard needs user
├─ Settings needs user
└─ Profile needs user
```

In this case, lifting state to a parent may still create prop drilling.

That is when Context API can help.

7. Context API

7.1 What Is Context API?

Simple definition:

Context lets components read shared data without passing props through every level.

Context is built into React.

You do not need to install a separate package to use it.

Context is useful for app-level or section-level values.

Examples:

```
theme
logged-in user
language
cart count
current organization
global UI preferences
```

7.2 What Problem Does Context Solve?

Without Context:

```
App -> Layout -> Header -> Navbar -> UserMenu
```

With Context:

```
App
├─ AuthProvider
│   └─ many components can read user directly
```

Context avoids passing the same prop through every level.

7.3 Context API Has Three Main Pieces

Context has three main pieces:

```
createContext() -> creates the shared container  
Provider       -> shares the value  
useContext()   -> reads the value
```

7.4 createContext()

`createContext()` creates the context object.

Example:

```
import { createContext } from "react";  
  
const ThemeContext = createContext(null);
```

Simple idea:

```
createContext creates the shared data container.
```

The `null` is the default value.

7.5 Provider

The Provider makes a value available to components below it.

Example:

```
<ThemeContext.Provider value={value}>  
  {children}  
</ThemeContext.Provider>
```

Simple idea:

Provider shares the value with everything inside it.

Important:

A component can only read the provided context value if it is inside the Provider.

7.6 useContext()

`useContext()` reads the context value.

Example:

```
import { useContext } from "react";  
  
const themeData = useContext(ThemeContext);
```

Simple idea:

`useContext` lets a child component read the shared value.

8. Full Context Example: Theme Toggle

This is the main Sprint 10 beginner project.

Goal:

Create a theme value.
Share it with Navbar and Home.
Toggle the theme without prop drilling.

Project structure:

```
src/  
  App.jsx  
  context/
```

```
ThemeContext.jsx  
components/  
  Navbar.jsx  
pages/  
  Home.jsx
```

8.1 File: `src/context/ThemeContext.jsx`

```
import { createContext, useContext, useState } from "react";  
  
const ThemeContext = createContext(null);  
  
export function ThemeProvider({ children }) {  
  const [theme, setTheme] = useState("light");  
  
  function toggleTheme() {  
    setTheme((currentTheme) =>  
      currentTheme === "light" ? "dark" : "light"  
    );  
  }  
  
  const value = {  
    theme,  
    toggleTheme,  
  };  
  
  return (  
    <ThemeContext.Provider value={value}>  
      {children}  
    </ThemeContext.Provider>  
  );  
}  
  
export function useTheme() {  
  return useContext(ThemeContext);  
}
```

8.2 Explanation of ThemeContext.jsx

Import React tools

```
import { createContext, useContext, useState } from "react";
```

This imports:

```
createContext -> creates context  
useContext    -> reads context  
useState      -> stores theme state
```

Create the context

```
const ThemeContext = createContext(null);
```

This creates the shared theme container.

Create the Provider component

```
export function ThemeProvider({ children }) {
```

`ThemeProvider` is a wrapper component.

It wraps the components that need theme access.

`children` means whatever is placed inside `ThemeProvider`.

Example:

```
<ThemeProvider>  
  <Navbar />  
  <Home />  
</ThemeProvider>
```

Here, `Navbar` and `Home` are the `children`.

Store the theme in state

```
const [theme, setTheme] = useState("light");
```

This means:

```
theme starts as "light"  
setTheme changes theme
```

Toggle the theme

```
function toggleTheme() {  
  setTheme((currentTheme) =>  
    currentTheme === "light" ? "dark" : "light"  
  );  
}
```

This means:

```
If current theme is light, change it to dark.  
If current theme is dark, change it to light.
```

The updater function is used because the new theme depends on the previous theme.

Create the shared value

```
const value = {  
  theme,  
  toggleTheme,  
};
```

This object contains what child components can read.

Child components can access:

```
theme  
toggleTheme
```

Provide the value

```
return (  
  <ThemeContext.Provider value={value}>  
    {children}  
  </ThemeContext.Provider>  
);
```

This shares `value` with all components inside the Provider.

Create a custom hook

```
export function useTheme() {  
  return useContext(ThemeContext);  
}
```

This is a custom hook.

Instead of writing this in every component:

```
useContext(ThemeContext)
```

You can write:

```
useTheme()
```

This makes the code cleaner.

8.3 File: `src/components/Navbar.jsx`

```
import { useTheme } from "../context/ThemeContext";  
  
export default function Navbar() {  
  const { theme, toggleTheme } = useTheme();  
  
  return (  
    <nav>  
      <p>Current theme: {theme}</p>  
    </nav>  
  );  
}
```

```

        <button type="button" onClick={toggleTheme}>
          Toggle theme
        </button>
      </nav>
    );
  }

```

8.4 Explanation of `Navbar.jsx`

```
const { theme, toggleTheme } = useTheme();
```

This reads the shared Context value.

`Navbar` does not receive props like this:

```
<Navbar theme={theme} toggleTheme={toggleTheme} />
```

Instead, it reads directly from Context.

This avoids prop drilling.

8.5 File: `src/pages/Home.jsx`

```

import { useTheme } from "../context/ThemeContext";

export default function Home() {
  const { theme } = useTheme();

  return (
    <main>
      <h1>Home Page</h1>
      <p>This page is using the {theme} theme.</p>
    </main>
  );
}

```

8.6 Explanation of Home.jsx

Home also reads the same theme value.

It does not need the theme passed as a prop.

This means both Navbar and Home can share the same theme state.

8.7 File: src/App.jsx

```
import { ThemeProvider } from "../context/ThemeContext";
import Navbar from "../components/Navbar";
import Home from "../pages/Home";

export default function App() {
  return (
    <ThemeProvider>
      <Navbar />
      <Home />
    </ThemeProvider>
  );
}
```

8.8 Context Tree Mental Model

```
App
├─ ThemeProvider
│   ├── Navbar -> reads theme and toggleTheme
│   └─ Home   -> reads theme
```

Important rule:

The Provider must wrap the components that need the context value.

If Navbar is outside ThemeProvider, it cannot read the provided theme value.

9. When Context API Is a Good Choice

Use Context when several components need the same shared value.

Good Context values:

```
theme
logged-in user
language
cart count
current organization
current project
global UI preferences
```

Example:

```
Navbar needs user
Dashboard needs user
Profile needs user
Settings needs user
```

This is a good reason to use Context.

10. When Context API Is Not a Good Choice

Context is not a replacement for all state.

Do not put every small value in Context.

Bad use cases:

```
Every input field in Context
Every button hover state in Context
Every small modal detail in Context
Every tiny local component value in Context
```

Better rule:

```
Use local state when one component owns the data.
Use props for normal parent-to-child data.
```

Lift state up when siblings need the same data.
Use Context when many distant components need the same shared value.

11. Context Warning: Unnecessary Re-renders

Context is useful, but it is not free.

When a Context value changes, components that read that Context may re-render.

Example:

```
<ThemeContext.Provider value={{ theme, toggleTheme }}>
```

If `theme` changes, components using `useTheme()` may re-render.

That is expected.

But if you put a fast-changing value in Context, many components may re-render too often.

Example of risky Context value:

```
A timer count that changes every second  
A mouse position that changes constantly  
A search input value that changes on every key press
```

If many components read that Context, many components may update often.

Beginner rule:

```
Use Context for stable shared app values.  
Be careful with frequently changing values.
```

12. Context API vs Props

12.1 Use Props When Data Flow Is Simple

Example:

```
function App() {
  return <UserCard name="Sangeeth" />;
}

function UserCard({ name }) {
  return <h2>{name}</h2>;
}
```

This is simple and clean.

No Context needed.

12.2 Use Context When Many Distant Components Need the Same Data

Example:

```
App
├─ AuthProvider
│   ├── Navbar reads user
│   ├── Dashboard reads user
│   ├── Profile reads user
│   └─ Settings reads user
```

This is a good use case for Context.

12.3 Simple Decision Rule

One child needs data:
Use props.

Sibling components need data:
Lift state up.

Many distant components need data:
Use Context.

13. External State Management Libraries

Context is built into React, but it is not the only state management option.

Sprint 10 introduces these tools:

```
Redux  
Redux Toolkit  
RTK Query  
Zustand
```

You do not need to master them yet.

At this stage, you need to understand why they exist.

14. Redux

14.1 What Is Redux?

Simple definition:

Redux is a state management library that stores shared state in a central store.

Redux is useful when an app has complex global state and many updates that must stay predictable.

14.2 Redux Mental Model

```
Component dispatches action  
↓  
Reducer receives current state and action  
↓  
Reducer returns next state  
↓  
Store updates  
↓  
UI reads updated state
```

14.3 Important Redux Terms

```
store    -> central state container  
action   -> object describing what happened  
reducer  -> function that calculates next state  
dispatch -> sends an action
```

14.4 Store

The store is the central place where Redux state lives.

Simple idea:

```
The store is the app's shared state container.
```

Example data in a store:

```
user  
cart items  
notifications  
settings  
permissions
```

14.5 Action

An action describes what happened.

Example action:

```
{  
  type: "cart/addItem",  
  payload: {  
    id: "p1",  
    name: "Keyboard"  
  }  
}
```

Simple idea:

An action is a message that says what happened.

14.6 Reducer

A reducer decides how state changes.

Example idea:

Current state + action = next state

A reducer should not mutate old state directly.

This is similar to what you learned with `useReducer`.

14.7 Dispatch

Dispatch sends an action.

Simple idea:

dispatch tells Redux that something happened.

Example:

```
dispatch({ type: "cart/addItem", payload: product });
```

14.8 When Redux Is Useful

Redux can be useful when:

many distant components need the same state,
state update rules are complex,
you need predictable state changes,
you need debugging tools for state changes,

you are working on a large team,
the app has complex business logic.

14.9 When Redux May Be Too Much

Redux may be unnecessary when:

the app is small,
only a few components share state,
Context is enough,
most state is local,
you are adding Redux only because it sounds professional.

Beginner warning:

Do not jump to Redux too early.

15. Redux Toolkit

15.1 What Is Redux Toolkit?

Redux Toolkit is the modern recommended way to write Redux logic.

It is often called RTK.

Simple definition:

Redux Toolkit makes Redux easier and less verbose.

15.2 Why Redux Toolkit Exists

Old Redux code could require a lot of repeated setup.

Redux Toolkit reduces boilerplate.

Simple idea:

Redux = powerful but can be verbose
Redux Toolkit = modern easier Redux

15.3 What Redux Toolkit Helps With

Redux Toolkit helps with:

creating stores,
creating reducers,
creating actions,
writing cleaner update logic,
organizing state slices,
reducing repeated code.

15.4 Beginner Rule

If you ever need Redux in a modern React app, learn Redux Toolkit instead of old manual Redux first.

16. RTK Query

16.1 What Is RTK Query?

RTK Query is part of Redux Toolkit.

It helps manage API/server data.

Simple definition:

RTK Query helps fetch, cache, refetch, and manage server data.

16.2 What Problem Does RTK Query Solve?

In Sprint 7, you learned this pattern:

```
loading
data
error
```

You may have written code like:

```
const [data, setData] = useState(null);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);
```

This is good for learning.

But in bigger apps, many pages may need API data.

Problems appear:

```
same API call repeated in many places,
manual loading state repeated,
manual error state repeated,
manual caching becomes hard,
refetching becomes hard,
keeping server data fresh becomes hard.
```

RTK Query helps solve these server-data problems.

16.3 RTK Query Helps With

```
fetching data
loading state
error state
caching
refetching
keeping API data organized
```

16.4 When RTK Query Is Useful

Use RTK Query when:

```
your app uses many API requests,  
you need caching,  
you need automatic refetching,  
you need organized server-state management,  
you are already using Redux Toolkit.
```

16.5 Client State vs Server State

Client state means data created and managed mainly in the frontend.

Examples:

```
theme  
modal open/closed  
selected tab  
local form draft  
sidebar expanded/collapsed
```

Server state means data that comes from an API/backend.

Examples:

```
products  
posts  
comments  
user profile from database  
orders  
notifications from server
```

Simple rule:

```
Redux/Zustand/Context often manage client state.  
RTK Query focuses on server state.
```

17. Zustand

17.1 What Is Zustand?

Zustand is a lightweight state management library.

Simple definition:

Zustand gives you a shared store using hooks, with less setup than Redux.

17.2 Zustand Mental Model

```
Create a store
↓
Put shared state and update functions in it
↓
Components use a hook to read or update the store
```

17.3 Why Developers Like Zustand

Zustand is popular because:

```
it has less boilerplate than Redux,
it is simple to start,
it uses hooks,
it can work without wrapping your app in a Provider for basic use,
it is useful for small to medium global state.
```

17.4 When Zustand Is Useful

Zustand can be useful for:

```
small to medium apps,
global UI state,
shared client-side state,
```

simple state stores,
state that feels too global for Context but too simple for Redux.

17.5 Zustand vs Redux Toolkit

Simple comparison:

Redux Toolkit:
More structured.
Good for complex apps and predictable state flow.

Zustand:
Lighter and simpler.
Good when you want global state with less setup.

18. Context API vs Redux Toolkit vs RTK Query vs Zustand

18.1 Comparison Table

Tool	Main Use	Best For	Beginner Warning
Local state	Component-owned data	One component	Do not lift too early
Lifted state	Shared sibling data	Nearby components	Can become prop drilling if overused
Context API	Simple shared app values	Theme, auth user, language	Not for every tiny state value
Redux Toolkit	Complex global client state	Large apps, predictable updates	More setup than Context
RTK Query	Server/API state	Fetching, caching, refetching	Learn basic fetch first
Zustand	Lightweight global state	Small/medium shared state	Still an external dependency

18.2 Beginner Decision Table

Situation	Use
One component needs the data	<code>useState</code>
Sibling components need the data	Lift state up
A parent passes data directly to a child	Props
Many distant components need simple shared data	Context API
Complex global state with many update rules	Redux Toolkit
API/server state with caching	RTK Query
Lightweight global store	Zustand

18.3 Default Beginner Order

When choosing a state solution, start simple.

```
useState
↓
lift state up
↓
props
↓
Context
↓
external state library only when needed
```

Do not start with Redux or Zustand unless the app actually needs it.

19. React DevTools

19.1 What Is React DevTools?

React DevTools is a browser tool that helps you inspect a React app.

Simple definition:

React DevTools lets you see your component tree, props, state, context, and render behavior.

It helps you debug React apps instead of guessing.

19.2 Main React DevTools Tabs

React DevTools has two very important tabs:

Components

Profiler

20. Components Tab

20.1 What Is the Components Tab?

The Components tab lets you inspect the React component tree.

Use it to answer questions like:

Which components are on the screen?

What props did this component receive?

What state does this component have?

What Context value is this component reading?

Where is this component in the tree?

Which component is the parent?

Which components are children?

20.2 Why the Components Tab Is Useful

When something is wrong in the UI, you can check:

Did the component receive the prop I expected?

Is the prop name correct?

Is the state value correct?

Is the Context value correct?

Is the component inside the Provider?

This is better than guessing.

20.3 Example: Inspecting Props

Code:

```
function Parent() {  
  return <Child message="Hello" />;  
}  
  
function Child({ message }) {  
  return <p>{message}</p>;  
}
```

In React DevTools, select `Child`.

You should see a prop like:

```
message: "Hello"
```

If the prop is missing, the parent may not be passing it correctly.

20.4 Example: Inspecting State

Code:

```
import { useState } from "react";  
  
function Counter() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <button type="button" onClick={() => setCount(count + 1)}>  
      Count: {count}  
    </button>  
  );  
}
```

In React DevTools, select `Counter`.

You can inspect its state value.

If the count does not update, check whether `setCount` is being called correctly.

20.5 Example: Inspecting Context

If a component uses Context:

```
const { theme } = useTheme();
```

React DevTools can help you check whether the component is receiving the expected context value.

If the context value is wrong, check:

```
Is the component inside the Provider?  
Is the Provider value correct?  
Is the custom hook reading the correct context?
```

21. Profiler Tab

21.1 What Is the Profiler Tab?

The Profiler tab helps you inspect rendering behavior.

Simple definition:

The Profiler helps you see which components rendered and how expensive the render was.

Use it when you want to answer:

```
Why did this component re-render?  
Which update was expensive?  
How long did a render commit take?  
Did Context cause many components to update?  
Did a parent re-render cause many child re-renders?
```

21.2 Beginner Profiler Workflow

Simple workflow:

```
Open React DevTools
Go to Profiler
Start recording
Click around in your app
Stop recording
Look at which components rendered
Check if any component rendered unexpectedly
```

For Sprint 10, you do not need advanced performance analysis.

You only need to learn how to observe re-renders.

21.3 What Is a Re-render?

A re-render means React runs a component function again to calculate the UI.

Re-renders are normal.

They happen when:

```
state changes,
props change,
a parent re-renders,
context value changes.
```

A re-render is not automatically bad.

The problem is unnecessary expensive work.

21.4 Context and Re-renders

When a Context value changes, components reading that Context may re-render.

Example:

```
Theme changes
↓
Navbar re-renders
```

Home re-renders
SettingsPanel re-renders

This is normal if those components use the theme.

But if many unrelated components re-render, you may need to rethink how your Context is structured.

22. Prop Mismatch Debugging

22.1 What Is a Prop Mismatch?

A prop mismatch happens when the parent passes one prop name, but the child expects a different prop name.

Example broken code:

```
function Parent() {  
  return <Child response="Hello" />;  
}  
  
function Child({ response }) {  
  return <Grandchild reply={response} />;  
}  
  
function Grandchild({ response }) {  
  return <h2>{response}</h2>;  
}
```

Problem:

Child passes: reply
Grandchild expects: response

So `Grandchild` receives `undefined` for `response`.

22.2 Fixed Version 1

Make the parent pass the same prop name the child expects.

```
function Parent() {
  return <Child response="Hello" />;
}

function Child({ response }) {
  return <Grandchild response={response} />;
}

function Grandchild({ response }) {
  return <h2>{response}</h2>;
}
```

Now the prop name is consistent:

```
response -> response -> response
```

22.3 Fixed Version 2

Or change the receiving prop name.

```
function Parent() {
  return <Child response="Hello" />;
}

function Child({ response }) {
  return <Grandchild reply={response} />;
}

function Grandchild({ reply }) {
  return <h2>{reply}</h2>;
}
```

Now the prop name is also consistent:

```
Child passes reply
Grandchild receives reply
```

Both fixes are valid.

The important rule is:

The prop name passed by the parent must match the prop name expected by the child.

22.4 How React DevTools Helps With Prop Mismatch

If a value is missing from the UI:

1. Open React DevTools.
2. Select the component that should display the value.
3. Check its props.
4. Confirm whether the prop exists.
5. Confirm whether the prop name is correct.
6. Move up to the parent component and check what it passed.

Debugging mental model:

```
UI is wrong
↓
Inspect child props
↓
Prop missing or undefined?
↓
Inspect parent
↓
Fix prop name or value
```

23. Sprint 10 Mini Project: Theme Toggle With Context

23.1 Project Goal

Build a small app where multiple components read the same theme value from Context.

Component tree:

```
App
├── ThemeProvider
│   ├── Navbar
│   ├── HomePage
│   └── SettingsPanel
```


Requirements:

Navbar shows current theme.
Navbar has a Toggle Theme button.
HomePage reads the theme.
SettingsPanel reads the theme.
No theme prop drilling.
Use React DevTools to inspect Provider and consumers.
Use Profiler after toggling theme.

Expected behavior:

Click Toggle Theme
↓
theme changes from light to dark
↓
Navbar, HomePage, and SettingsPanel update
↓
No prop drilling required

23.2 Suggested File Structure

```
src/  
  App.jsx  
  context/  
    ThemeContext.jsx  
  components/  
    Navbar.jsx  
    SettingsPanel.jsx  
  pages/  
    HomePage.jsx
```

23.3 ThemeContext.jsx

```
import { createContext, useContext, useState } from "react";  
  
const ThemeContext = createContext(null);  
  
export function ThemeProvider({ children }) {
```

```

const [theme, setTheme] = useState("light");

function toggleTheme() {
  setTheme((currentTheme) =>
    currentTheme === "light" ? "dark" : "light"
  );
}

return (
  <ThemeContext.Provider value={{ theme, toggleTheme }}>
    {children}
  </ThemeContext.Provider>
);
}

export function useTheme() {
  return useContext(ThemeContext);
}

```

23.4 Navbar.jsx

```

import { useTheme } from "../context/ThemeContext";

export default function Navbar() {
  const { theme, toggleTheme } = useTheme();

  return (
    <nav>
      <strong>Theme: {theme}</strong>
      <button type="button" onClick={toggleTheme}>
        Toggle Theme
      </button>
    </nav>
  );
}

```

23.5 HomePage.jsx

```

import { useTheme } from "../context/ThemeContext";

export default function HomePage() {

```

```
const { theme } = useTheme();

return (
  <main>
    <h1>Home Page</h1>
    <p>The current theme is {theme}</p>
  </main>
);
}
```

23.6 SettingsPanel.jsx

```
import { useTheme } from "../context/ThemeContext";

export default function SettingsPanel() {
  const { theme } = useTheme();

  return (
    <section>
      <h2>Settings</h2>
      <p>Theme setting: {theme}</p>
    </section>
  );
}
```

23.7 App.jsx

```
import { ThemeProvider } from "../context/ThemeContext";
import Navbar from "../components/Navbar";
import SettingsPanel from "../components/SettingsPanel";
import HomePage from "../pages/HomePage";

export default function App() {
  return (
    <ThemeProvider>
      <Navbar />
      <HomePage />
      <SettingsPanel />
    </ThemeProvider>
  );
}
```

```
);  
}
```

23.8 Mini Project Self-Check

Your app is correct if:

- `Navbar` shows the current theme.
- Clicking the button changes the theme.
- `HomePage` updates when the theme changes.
- `SettingsPanel` updates when the theme changes.
- You did not pass `theme` through props.
- The components read theme through `useTheme()`.
- `ThemeProvider` wraps the components that need theme.

24. Common Beginner Mistakes

Mistake 1: Using Context for Everything

Bad:

Putting every small input value into Context.

Better:

Keep local state local.

Mistake 2: Forgetting the Provider

Broken mental model:

```
export default function App() {  
  return <Navbar />;  
}
```

But `Navbar` uses `useTheme()`.

If `Navbar` is not inside `ThemeProvider`, the context value may be missing.

Correct:

```
export default function App() {  
  return (  
    <ThemeProvider>  
      <Navbar />  
    </ThemeProvider>  
  );  
}
```

Mistake 3: Prop Name Mismatch

Broken:

```
<Grandchild reply={response} />  
  
function Grandchild({ response }) {  
  return <h2>{response}</h2>;  
}
```

Problem:

```
Passed prop: reply  
Expected prop: response
```

Fix:

```
<Grandchild response={response} />
```

Mistake 4: Jumping to Redux Too Early

Bad reasoning:

```
This app has state, so I need Redux.
```

Better reasoning:

```
Can local state solve it?  
Can lifted state solve it?  
Can Context solve it?  
Only then consider Redux Toolkit or Zustand.
```

Mistake 5: Ignoring React DevTools

Bad debugging:

```
Guess where the prop disappeared.
```

Better debugging:

```
Open React DevTools.  
Inspect the component.  
Check props, state, and context.
```

25. Sprint 10 Debug Task

Broken Code

```
function Child({ response }) {  
  return <Grandchild reply={response} />;  
}  
  
function Grandchild({ response }) {  
  return <h2>{response}</h2>;  
}
```

Problem

Child passes a prop named reply.
Grandchild expects a prop named response.

So `Grandchild` receives `undefined` for `response`.

Fixed Code Option 1

```
function Child({ response }) {  
  return <Grandchild response={response} />;  
}  
  
function Grandchild({ response }) {  
  return <h2>{response}</h2>;  
}
```

Fixed Code Option 2

```
function Child({ response }) {  
  return <Grandchild reply={response} />;  
}  
  
function Grandchild({ reply }) {  
  return <h2>{reply}</h2>;  
}
```

Debug Rule

The prop name passed by the parent must match the prop name expected by the child.

26. Sprint 10 Revision Checkpoint Questions

Answer these without looking at the notes.

1. What is prop drilling?
2. Why does prop drilling become hard to maintain?
3. Is all prop passing bad?
4. What does “single source of truth” mean?
5. Why is single source of truth useful?
6. When should state stay local?
7. When should you lift state up?
8. When is Context better than normal props?
9. What are the three main pieces of Context API?
10. What does `createContext()` do?
11. What does the Provider do?
12. What does `useContext()` do?
13. Why is a custom context hook like `useTheme()` useful?
14. What kind of values are good for Context?
15. What kind of values should usually not go into Context?
16. Why can Context cause unnecessary re-renders?
17. What problem does Redux solve?
18. What is a Redux store?
19. What is a Redux action?
20. What is a Redux reducer?
21. What does dispatch mean?
22. Why does Redux Toolkit exist?
23. What problem does RTK Query solve?
24. What kind of state is server state?
25. What kind of state is client state?
26. What makes Zustand lighter than Redux?
27. Which React DevTools tab shows props, state, and context?
28. Which React DevTools tab helps investigate re-renders?
29. What is a prop mismatch?
30. How can React DevTools help debug a prop mismatch?

27. Sprint 10 Practice Tasks

Task 1: Explain Prop Drilling

Write a short explanation in your own words:

Prop drilling means...

Then draw this tree:

```
App -> Layout -> Header -> Navbar -> UserMenu
```

Mark which component actually uses the data.

Task 2: Build Theme Context

Build the full Theme Toggle project.

Requirements:

- Create `ThemeContext.jsx`.
 - Create `ThemeProvider`.
 - Create `useTheme()`.
 - Create `Navbar`.
 - Create `HomePage`.
 - Create `SettingsPanel`.
 - Wrap everything in `ThemeProvider`.
 - Toggle theme with a button.
-

Task 3: Inspect with React DevTools

After building the project:

1. Open React DevTools.
 2. Open the Components tab.
 3. Select `Navbar`.
 4. Check what values it receives.
 5. Select `HomePage`.
 6. Confirm it reads the theme.
 7. Toggle the theme.
 8. Watch the values update.
-

Task 4: Use the Profiler

1. Open the Profiler tab.
2. Start recording.
3. Click the Toggle Theme button.
4. Stop recording.
5. Look at which components rendered.

6. Ask: did these components need to render?

Task 5: Fix a Prop Mismatch

Fix this code:

```
function Parent() {  
  return <Child message="Hello" />;  
}  
  
function Child({ message }) {  
  return <Grandchild text={message} />;  
}  
  
function Grandchild({ message }) {  
  return <p>{message}</p>;  
}
```

Possible fix:

```
function Parent() {  
  return <Child message="Hello" />;  
}  
  
function Child({ message }) {  
  return <Grandchild message={message} />;  
}  
  
function Grandchild({ message }) {  
  return <p>{message}</p>;  
}
```

28. Sprint 10 Definition of Done

You are done with Sprint 10 when you can:

- explain prop drilling in simple words,
- identify prop drilling in a component tree,
- explain single source of truth,
- choose between local state, lifted state, props, and Context,
- create a Context with `createContext()`;

- wrap components with a Provider,
- read Context using `useContext()` or a custom hook,
- explain when Context is useful,
- explain when Context is not a good choice,
- explain what Redux is for,
- explain why Redux Toolkit exists,
- explain what RTK Query is for,
- explain what Zustand is for,
- use React DevTools Components tab to inspect props, state, and context,
- use React DevTools Profiler tab to observe re-renders,
- fix a prop mismatch bug.

29. Final Beginner Mental Model

Sprint 10 teaches this:

In small React apps, props and local state are usually enough.

As the app grows, passing props through many levels becomes messy.

That problem is prop drilling.

Context API lets shared values like theme or logged-in user be read by deep components without passing props through every level.

Context is useful, but not for every state value.

For complex global state, Redux Toolkit or Zustand may be better.

For API/server data with caching, RTK Query may be useful.

React DevTools helps inspect props, state, context, and re-renders so you can debug instead of guessing.

Simple final rule:

Start simple.

Use local state first.

Lift state when needed.

Use Context for simple shared app values.

Use external state tools only when the app becomes complex enough to need them.

Use React DevTools to verify what is happening.